Transformer 前置知识

> 一份零基础友好的 Transformer 入门指南。读完你就能看懂 QKV、多头注意力、位置编码、FFN、

> 训练推理差异、分词机制，然后再去读横纵分析报告就会顺畅很多。

>

> **撰写时间**：2026-04-29 | **最后更新**：2026-04-30（补充推理优化、量化、RAG、Hyperscale Lottery）

目录

1. 为什么要懂 Transformer?
2. 前 Transformer 时代：RNN 帝国为什么倒了
3. 注意力机制：Query、Key、Value 到底是什么
4. 多头注意力：为什么需要八个头
5. 位置编码：为无序的计算注入顺序
6. FFN 层：被低估的记忆仓库
7. Encoder、Decoder、Encoder-Decoder：三大架构的来龙去脉
8. 为什么 Transformer 能 scale
9. Scaling Laws：从 Kaplan 到 Chinchilla
10. 训练与推理：Transformer 的双面人生
11. 分词：Transformer 怎么「识字」
12. 激活函数与归一化：被忽略的"螺丝钉"
13. 注意力复杂度危机与 FlashAttention
14. 注意力压缩三部曲：MQA → GQA → MLA
15. MoE-Transformer 入门：把 FFN 拆成专家团队
16. 长上下文与推理优化
17. 训练精度：FP16、BF16、FP8 和混合精度
18. In-context Learning 与推理时计算
19. 后训练：RLHF、DPO 与对齐
20. 替代架构探秘：SSM、Mamba 与混合路线

21. 多模态 Transformer：ViT 与图文模型
22. 一张图谱串起来
23. 与 MoE 前置知识的对照
24. 延伸阅读

一、为什么要懂 Transformer?

2026 年，你用的每一个 AI 产品背后几乎都站着 Transformer——GPT-4、Claude、Gemini、DeepSeek、文心一言、通义千问。它不只是大语言模型的基础，还是视觉（ViT）、语音（Whisper）、蛋白质折叠（AlphaFold）等 AI 前沿的通用架构。

这篇文章的目标读者是**没有深度学习背景的人**。全部用大白话讲，不讲数学推导，只说直觉和关键概念。

在开始之前，先搞懂三个最基础的概念。还有一个贯穿全文的词——"层"：

层就是一道加工工序。Transformer 有几十到上百层，每一层对输入做一次小加工——就像流水线。第 1 层打磨一下，传给第 2 层继续打磨……最后一层出来的就是成品。层越多，能做的加工越精细。

- **模型 = 一堆可调整的数字（参数），一层层堆起来，把输入变成输出。** 就像一台计算器：你在左边输入中文，它从右边输出下一句中文。数字越大（比如 700 亿个），这台计算器越"聪明"
- **训练 = 不断微调这些数字。** 给模型看一道题→它算出答案→跟正确答案比差距→往差距变小的方向拧每一个数字。拧几十亿次，模型就越来越准。这个差距叫 loss，越大说明错得越离谱
- **推理 = 不再拧数字，只管算。** 输入→逐层计算→输出结果。不调数字，只跑一遍。这就是你每次跟 ChatGPT、Claude 聊天时发生的事

整篇文章都在讲这三件事的"怎么做"和"为什么这样做"。在此基础上，读完之后你会理解：

- Transformer 跟以前的 RNN/LSTM 到底区别在哪
- Q、K、V 是什么，为什么计算注意力时要缩放防止数字爆炸
- 为什么需要那么多"头"
- Encoder、Decoder、Encoder-Decoder 三大架构各自为什么存在
- Scaling Laws 是什么，Kaplan 和 Chinchilla 为什么结论差了 10 倍
- 训练和推理的本质差异，KV Cache 怎么加速
- FlashAttention 为什么没改算法却能快 4 倍
- MQA → GQA → MLA 这条压缩线为什么会发生
- MoE 怎么把 FFN 拆成专家团队
- INT8/INT4 量化怎么做，GGUF/GPTQ/AWQ 有什么区别
- Continuous Batching、Prefix Caching、vLLM 怎么加速推理
- 分词这个小细节为什么能决定模型天花板

文中会不断跟 **MoE（混合专家模型）** 做对比参照——因为两者共享一个核心思路：**不把所有事情都塞给所有人，而是根据"谁更擅长什么"来分配任务**（在 Transformer 里叫软路由，MoE 里叫专家分工）。对照着看会豁然开朗。

关键术语速查

刚开始看难免遇到生词，这里先列一遍，后面的讲解都会用到：

术语	大白话解释
模型	一堆可调整的数字（参数），一层层堆起来，把输入变成输出。LLaMA 70B = 700 亿个数字、80 层堆叠
参数	模型里可调整的数字。70B 参数 = 700 亿个数字。参数越多，"脑子容量"越大，但越吃显存
训练	给模型看数据→算预测→对比标准答案→往 loss 变小的方向微调每个参数。重复几十亿次
推理	训练完的模型不再调参数，只是输入→层层计算→输出。跟训练的差别就是"不调了，只算"
学习率（Learning Rate）	训练时每次调整参数的"步长"。太大 = 跳过头，太小 = 永远学不完。最重要的训练超参数
前向 / 前向传播	输入数据经过模型层层计算、输出结果的过程。不调参数，只是"算一遍"。跟"反向传播"对应
Epoch	把整个训练数据集完整过一遍。15T tokens ÷ 8B 参数 ≈ 每 epoch ~1.6T tokens，LLaMA 3 8B 训了约 9 个 epoch
Scaling（扩规模）	让模型变得更强的通用方法：更多参数、更多数据、更多算力。投入翻倍→性能可预测地提升。Transformer 最大的护城河
序列建模	处理有顺序的数据（文本、语音、DNA）的任务。输入有前后关系，不能打乱。前 Transformer 时代的核心问题
Zero-shot / Few-shot	Zero-shot=不给示例直接做任务。Few-shot=给几个示例再做。GPT-3 发现的 in-context learning 能力，彻底改变了大语言模型（LLM）的使用方式
In-context Learning	不需要重新训练，把任务说明和几个例子直接写在提示词（prompt）里，大模型就能照做。GPT-3 时代最重要的发现之一
Chain-of-Thought（CoT）	让模型在给出答案前先"一步步思考"，推理准确率大幅提升。2025 年后成为"推理时扩规模"的核心手段
RLHF / DPO	让模型学会"什么该说什么不该说"的技术。RLHF=人给回答打分→教模型写高分答案。DPO=直接对比好回答和坏回答的差距来训练，更简单
SSM（State Space Model）	替代 Attention 的一种方案。复杂度更低（输入翻倍时算量也只翻一倍，不像 Transformer 翻四倍）。Mamba 是代表。2025 年后成为混合架构的主流组件
蒸馏（Distillation）	用大模型教小模型。大模型生成"标准答案"，小模型模仿学习。Phi-3 和 Gemma 2 的核心技术
TTFT（首 Token 延迟）	Time To First Token。从发送请求到收到第一个输出 token 的时间。在线服务最核心的延迟指标
GRPO	DeepSeek-R1 用的强化学习算法。不需要外部奖励模型，用同一批问题的多个回答互相对比来训练推理能力
合成数据	用 AI 生成的训练数据。高质量真实数据耗尽后，合成数据成为继续扩大模型能力的关键替代方案
Adam / AdamW	最主流的训练调节算法。负责控制"每次拧参数的力度"。AdamW 加了权重衰减避免数字长得太大，几乎所有 LLM 都用它训练

Warmup（学习率预热）	训练开始时先把学习率从小逐渐升到目标值。避免初期训练不稳定导致崩溃
NSP（Next Sentence Prediction）	BERT 的辅助训练任务：判断两句话是否前后相连。RoBERTa 证明这个任务贡献很小，后来的模型都不用了
DualPipe	DeepSeek V3 的并行算法。让前向/反向计算与 MoE 跨节点通信完全重叠，MFU 因此达到 ~70%
填充中间（FIM）	Fill-in-the-Middle。代码模型的特殊训练格式：`<PRE>前缀 <SUF>后缀 <MID>补全` 。StarCoder/CodeLlama 都在用
BLEU	机器翻译的质量评分，0-100 分。越接近人工翻译的分数越高。Transformer 原始论文达到 28.4 分（当时最高）
Label Smoothing	训练技巧：不要求模型 100% 确定选某个词，留一点"不确定性"。值 0.1 表示正确答案概率=0.9，其余均分。防过拟合
vLLM	最流行的 LLM 推理部署框架。核心创新 PagedAttention（把 KV cache 当虚拟内存分页管理）。吞吐量比普通推理高 2-4 倍
对比学习	训练方法：让匹配的样本（图文对）向量尽可能接近，不匹配的尽可能远离。CLIP 和检索系统的基础
ViT（Vision Transformer）	Transformer 的图像版本。把图片切成 16×16 的小块（patch），每个 patch 当成一个 token 处理
CNN（卷积神经网络）	前 Transformer 时代的图像处理主流架构，用滑动窗口局部处理。ViT 的全局视野更完整
显存（VRAM）	GPU 上专门的高速内存。存模型参数 + 中间结果。H100 一张 80GB——一个 LLaMA 70B 就要 140GB
泛化	在训练数据上学到通用规律，遇到从没见过的数据也能正确处理。大 batch 可能降低泛化
浮点数	带小数点的数字（如 3.14=FP32）。FP32=32位、FP16=16位、FP8=8位。位数越少越省空间和算力
交叉熵	最常用的 loss 计算方法。衡量两个概率分布之间的"距离"——预测分布和真实标签之间差多少
收敛	训练中 loss 不再下降的状态。说明模型"学到头了"或"学不动了"
预训练（Pre-training）	在巨量通用数据上先训练出一个基础模型，学会语言通用规律。比如 GPT 先在海量网页上训练
微调（Fine-tuning）	在预训练好的模型上，用小量特定数据继续训练。比如让它学会写医疗报告
Loss（损失函数）	训练时衡量"预测与标准答案差多少"的数字。loss 越大模型越错，训练目标就是让 loss 越来越小
Batch size（批量大小）	每次训练同时喂给模型的样本数。batch 越大训练越快，但越吃显存，超大 batch 还可能降低泛化
Linear layer（线性层）	矩阵乘法变换，FFN 和 Embedding 的基本操作。输入一串数字，经过矩阵乘法输出另一串数字

Token	文本的最小处理单元。不是单词——"transformer"可能被切成 3 个 token："trans"+"form"+"er"
Embedding	把 token 变成一串数字（向量），让计算机能算。语义相近的词向量也相近
向量	就是一串数字。比如"猫"的 embedding 是 [0.2, −0.5, 0.8, ...]，有 512 或 4096 个数字
Encoder（编码器）	"读懂输入"的模块——吃进一句话，输出每个词的理解表示
Decoder（解码器）	"写出输出"的模块——根据编码器的理解，一个字一个字往外生成
梯度	训练时告诉模型"往哪个方向调参数"的信号。梯度消失 = 信号越来越弱学不动。梯度爆炸 = 信号突然巨大导致崩溃
反向传播	算出 loss 之后，从输出层倒着往前传，通知每层"你该往哪调"。RNN 传太远信号就没了
Softmax	把任意一串数字变成概率分布（值在 0–1 之间、加起来等于 1）
d_k / d_model	向量维度。d_model=512 意思是每个 token 用 512 个数字表示
O(n) / O(n^2)	"计算量随输入长度增长的速度"。O(n^2) 表示输入翻倍，算量翻四倍
残差连接	把一层的输入直接加到输出上（输出 = 原输入 + 本层计算结果）。防止深层网络梯度消失
LayerNorm（层归一化）	把一层的输出"拉回标准范围"，均值为 0、方差为 1，训练更稳定
激活函数	FFN 里的"开关"——输入够大就放行，不够大就压住。没有它再多层也等于一层
Ground Truth	训练数据里的标准答案。比如输入"the cat"，GT 就是"sat"
[BOS] / [EOS]	特殊 token。Begin of Sequence = 句子开头，End of Sequence = 句子结束
HBM / SRAM	GPU 显存的两个层级。HBM 容量大但慢（放模型参数），SRAM 小但快 10 倍（放临时计算）
FP16 / BF16 / FP8	浮点数精度格式。位数越少模型越小越快，但可能溢出或精度不够
INT8 / INT4	整数量化精度。用于推理时压缩模型，精度损失 <1% 时基本无感
MFU	模型算力利用率。GPU 实际干活时间 ÷ 总运行时间的比例。DeepSeek V3 达到 ~70%，GPT-4 约 32%
TFLOPS / PFLOPS	浮点运算速度单位。T=万亿次/秒，P=千万亿次/秒。衡量 GPU 算力的标准指标

二、前 Transformer 时代：RNN 帝国为什么倒了

2017 年之前，序列建模是 **RNN（循环神经网络，Recurrent Neural Network）** 的天下。它的原理很直观——一条句子进来，从左到右逐个 token 吞下，每读一个词就更新自己的"记忆状态"，把当前理解传给下一步。

打个比方：RNN 像个在听句子的人，每听一个词就在心里默念一遍之前的摘要，最后把全部摘要合并成对整句的理解。

RNN 有两个升级版：**LSTM（长短期记忆网络，Long Short-Term Memory）** 和 **GRU（门控循环单元，Gated Recurrent Unit）**。它们给 RNN 加了"门控机制"——相当于多了几个开关，决定"哪些信息保留、哪些信息忘掉"——能记住 20 个 token 之前"主语是单数"这种语法信息了。

但有个绕不过去的硬伤：**串行计算**。

要算第 100 个词的"心里摘要"，必须先把前面 99 步都跑完。GPU 几千个核心，RNN 只能用上一个。训练一条长句子，大部分时间 GPU 在空转。更致命的是，虽然 LSTM 能记住 20 步前，到 100 步、200 步，该忘还是忘——反向传播路径太长，梯度信号被洗成噪声，模型学不到长距离的规律。

Seq2Seq 架构（Sutskever et al., 2014）给这个问题打了一个补丁，也是后来 Transformer 的直属前身：用 Encoder 把整个源句压缩成一个定长向量（一串数字），Decoder 从这串数字解码出目标句。问题显而易见——定长压缩本身就是信息瓶颈。句子一长，前半句的信息被后面的内容挤掉了。

注意力机制（Bahdanau et al., 2014）是第一个真正的突破：不再用一个定长向量代表全句，Decoder 每一步都**动态搜索**整个源句，自己去挑"现在该看哪些词"。比如生成法语动词时，模型自动去源句里找主语在哪。

这是关键的思想跃迁——从"被动接收一个压缩表示"变成了"主动检索"。这个"软寻址"的视角，后来成了理解 Transformer 的核心隐喻。

> **MoE 对照**：MoE 预训练的历史同样经历了"从全量计算到条件计算"的跃迁——从所有样本经过所有参数（Dense），到每个 token 只激活部分专家。两者底层逻辑完全一致：**不再全体接收信息，而是按需挑选**。

三、注意力机制：Query、Key、Value 到底是什么

自注意力（Self-Attention）是 Transformer 的心脏。

一句话说明白：**让每个 token 去问其他 token："你跟我什么关系？"然后根据关系强弱，加权汇总其他 token 的信息。**

Q、K、V 是三个可训练的权重矩阵（你可以理解成三套"配方"，训练过程中自动调优）：

- **Query（查询）**：当前 token 在问什么。比如"我是动词 'eat'，我需要知道谁在吃、吃了什么"
- **Key（键）**：每个 token 提供什么信息标签。比如"我是名词，我可以当主语"
- **Value（值）**：每个 token 携带的实质信息。比如"我的具体语义是 'cat'"

计算过程：

```
Attention(Q, K, V) = softmax(QK^T / √d_k) × V
```

分四步理解：

1. **QK^T**：拿当前 token 的 Query 跟所有 token 的 Key 做点积（相似度计算），得到一个"关注度分数矩阵"
2. **÷ √d_k**：除以 Key 维度的平方根。因为维度一大，点积值会爆，softmax 后面的梯度趋近于零，模型学不动了。除以 √d_k 把方差控制回 1

3. **softmax**：把关注度分数变成概率（0–1 之间，所有加起来等于 1）
4. **× V**：用这个概率分布去加权汇总所有 token 的 Value——"谁跟我越相关，谁的信息我就听得越重"

这个视角跟数据库检索一模一样：Query 是搜索词，Key 是文档索引，Value 是文档内容。跟 MoE 的 Router 也很像——MoE 用门控网络决定"每个 token 去哪几个 expert"，Transformer 用 QK 相似度决定"每个 token 关注哪几个 token"。都是**按需分配**，只是分配的对象不同（Transformer 分配信息，MoE 分配计算资源）。

一张图理解 QKV（小白视角）

想象你要翻译："The cat sat on the mat because it was tired."

当模型处理"it"时：

```
"it" 的 Query = "我正在找一个单数名词，我可能是代词指代"
所有 token 的 Keys = ["The": 冠词, "cat": 单数名词, "sat": 动词, ...]
"cat" 的 Key 跟 "it" 的 Query 匹配度最高 → Attention 分数最大
"cat" 的 Value = "猫" → 被大部分权重传递给 "it"
```

结果：模型"知道"了 it 指的是 cat，而不是 mat。这个能力叫**指代消解**（Coreference Resolution），在传统 NLP 里是一个需要单独建模的任务，Transformer 把注意力做到位了就自然而然解决了。

Cross-Attention vs Self-Attention

- **Self-Attention**：序列内部 token 互看。Encoder 能看整句，Decoder 只能看已生成部分（不能偷看未来）
- **Cross-Attention**：Decoder 的 Query 去检索 Encoder 所有位置的 Key/Value，问"源句里哪个词跟我要生成的下一个词最相关？"

Decoder-only 架构（GPT 系）能成为主流，正是因为把 Cross-Attention 省掉了——所有知识都包装在 Self-Attention + FFN 里，推理时一个模型从头跑到尾。

四、多头注意力：为什么需要八个头

单头注意力的表达能力有限——一个 token 在一次前向中只能形成一种"关系模式"。但实际上，一个 token 可能需要同时关注"语法结构"（动词接什么介词）和"语义角色"（谁是施事者）。

多头注意力的做法：把 512 维的大向量切成 8 个 64 维的小向量，每个小向量独立计算注意力，最后拼接起来过一层线性变换：

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) × W_O
```

不同头确实学会了关注不同的关系模式。有的头负责相邻词之间的句法依赖，有的头负责长距离语义关联，有的头专门看标点符号。这不是设计出来的——是训练过程的涌现行为。

> **MoE 对照**：多头注意力很像"在注意力维度上做了细粒度专家"——每个头是一个专门的关系检测器，并行工作、最后融合。这也解释了为什么后来 DeepSeek 的 MLA 能用低秩压缩减少 KV cache：多个头之间的信息有大量重叠，可以被压缩进更小的潜空间。这和 MoE 把一个大 FFN 拆成多个小 expert 的"细粒度"思路异曲同工。

五、位置编码：为无序的计算注入顺序

RNN 天然知道顺序——按序输入。Transformer 并行喂入所有 token，序列顺序完全丢失了。"我爱北京天安门"和"门安天京北爱我"对这层矩阵来说没区别。

位置编码就是给每个 token 打上"我是第 i 个"的标签。四代方案演进：

方案	做法	优点	缺点
Sinusoidal（原始论文）	sin/cos 函数编码位置	理论优雅，可线性外推	实际效果一般
Learned（BERT 用）	可训练的 embedding 查表	简单有效	无法外推到训练长度之外
RoPE（Su et al., 2021）	对 Q、K 向量做旋转	天然编码相对距离，外推性好	长序列外推仍需调整
ALiBi（Press et al., 2021）	直接在注意力分数上加偏置	极简，外推性最好	精度略低于 RoPE

RoPE 为什么赢？ 它不对输入向量加位置编码，而是对 Q 和 K 向量做旋转——旋转角度由位置决定。这样点积值天然包含相对位置信息（旋转矩阵的性质：Rot(m)^T Rot(n) = Rot(n−m)）。LLaMA、Mistral、Qwen、DeepSeek 全都采用了 RoPE。

外推性是指模型在训练时只见过 4K 长度的序列，推理时却能处理 8K 甚至更长的序列。RoPE 和 ALiBi 的外推性都比 Sinusoidal 好得多，这也是它们能取代原始方案的关键。

六、FFN 层：被低估的记忆仓库

每层 Transformer 有两个子层：Multi-Head Attention 和 Feed-Forward Network（前馈网络，FFN）。大家在聊 Attention，很少聊 FFN。

但 **FFN 占了 Transformer 总参数的三分之二**。Geva et al.（2021）给出过一个漂亮的观点——FFN 本质上是一个 key-value 记忆库：

```
第一层线性变换 W1：检测"现在输入里有什么模式？"
激活函数（ReLU/GELU/SwiGLU）：阈值开关，"这个模式激活了吗？"
第二层线性变换 W2：输出"这个模式对应的知识"
```

Transformer 的两个子层分工明确：

- **Attention** 回答"这段上下文里说了什么？"——从当前输入检索信息
- **FFN** 回答"根据你的训练知识，接下来该是什么？"——从参数记忆提取知识

> **MoE 的切入点恰恰在这里。** 既然 FFN 是一个记忆库，为什么每个 token 都要查询整个记忆库？能不能让 token 只查最相关的几个"记忆分区"？这就是 MoE-Transformer 的核心动机：把 FFN 拆成多个 expert，每个 token 通过门控网络只激活 1-2 个 expert。计算量不变，总参数量可以大幅增加——模型"知道"的更多，"花的算力"一样。

七、Encoder、Decoder、Encoder-Decoder：三大架构的来龙去脉

很多人学完 Transformer 以为只有一种架构。实际上原始论文发布后，迅速分化成了三个家族。理解这个分歧，是理解整个 LLM 行业格局的前提。

原始 Transformer（2017）：Encoder-Decoder

原始论文是为机器翻译设计的——Encoder 读英语，Decoder 写德语。Encoder 用双向 Self-Attention（每个词看整句），Decoder 用单向 Masked Self-Attention（只看左边）+ Cross-Attention（看 Encoder 的输出）。

这种设计天然适合"输入 A → 输出 B"的任务，但推理时需要先跑完整句 Encoder，再逐步 Decoder——两个模型都占用显存。

Encoder-Only：BERT 系（2018-至今）

Google 发现：如果只做理解（分类、问答、NER（命名实体识别——比如从句子中找出人名、地名、组织名）），Decoder 是多余的。把 Decoder 砍掉、只留 Encoder，用双向上下文做 Masked Language Model 预训练——遮一个词，从左右两个方向同时猜。

优势：双向上下文是语言理解的天然最优解。同一个词在不同语境下的表示完全不同（苹果 = 水果 vs 苹果 = 公司），这是单向模型做不到的。

劣势：不能生成文本。BERT 被限制在理解类任务里，而生成才是更通用的元能力。

代表：BERT → RoBERTa → DeBERTa → ModernBERT（2024，用 RoPE + FlashAttention 做了现代化改造）

Decoder-Only：GPT 系（2018-至今）

OpenAI 从第一天就以"生成"为目标。把 Encoder 也砍掉，只留 Decoder——自回归语言模型，逐字续写。

优势：架构最简洁——推理只需要一个模型，不依赖 Encoder 编码。生成是最通用的能力——能生成自然也会理解。

劣势：单向上下文（只看左边），理解能力理论上弱于双向。但这个劣势在足够大的规模下被抹平了。

为什么最终胜出？ 2018-2020 年，BERT 系在理解任务上完胜（GLUE（通用语言理解评测，当时 NLP 最权威的考试）榜单被 BERT 系模型刷屏），GPT 系在生成任务上默默积累。但到了 2020 年 GPT-3 证明了——一个足够大的 Decoder-Only 模型，不需要 fine-tune 就能完成分类、翻译、摘要等理解任务。生成是更高层级的元能力。

代表：GPT-1 → GPT-2 → GPT-3 → LLaMA → Mistral → Qwen → DeepSeek → Claude

为什么现在是 Decoder-Only 的天下?

维度	Encoder-Only	Encoder-Decoder	Decoder-Only
适合任务	理解（分类、检索）	翻译、摘要	通用生成
上下文	双向	Encoder双向 + Decoder单向	单向
推理成本	低	高（两个模型）	中
Scaling	有限	有限	**无限**
当前地位	专用	被替代	**主流**

Decoder-Only 的胜出不是因为它最好，而是因为它**最能 scale**——架构最简洁、推理最高效、生态最统一。

八、为什么 Transformer 能 scale

Scaling 的前提是三个能力，少一个都不行：

1. **完全并行**：训练时所有 token 同时进入，GPU 全部核心都在干活。对比 RNN——1024 个 token 要串行 1024 步——这是代际差异。
1. **恒定路径长度**：任意两个 token 之间的信息只需 O(1) 层 attention 就能直达。RNN 要一步步传过整条序列，梯度衰减严重。这是长距离依赖能力的根因。
1. **结构极度规整**：所有层结构相同、所有位置处理方式相同。想加参数？加层、加宽、加头——没有架构瓶颈挡路。RNN 同样可以堆叠，但梯度消失限制了深度；Transformer 的残差连接 + LayerNorm 让几百层的网络也能稳定训练。

这三个能力叠加，加上下节要讲的 Scaling Laws，让 Transformer 成了第一个"投入更多算力就能稳定变强"的架构。对大公司来说，这就是"投钱就有效果"——一个可预测的增长曲线比任何架构创新都更有商业价值。

九、Scaling Laws：从 Kaplan 到 Chinchilla

Scaling Laws（缩放定律） 是 AI 行业近五年来最重要的实证发现之一：模型性能（以 loss 衡量）随模型参数量 N、训练数据量 D、算力 C 的增加，遵循平滑的**幂律关系**。幂律的意思是——三者中任何一个翻倍，loss 都按可预测的幅度下降。

Kaplan（OpenAI, 2020）：参数优先

Kaplan 等人首次系统性地研究了这三个变量之间的关系。他们的核心结论：

> 增加参数比增加数据更有效——73% 的额外算力应该分配给更大的模型，只有 27% 给更多数据。

公式近似：N_opt ∝ C^0.73，D_opt ∝ C^0.27。

直接影响：GPT-3（175B 参数，仅 ~300B tokens 训练，约 1.7 tokens/参数）和 Gopher（280B 参数，300B tokens）都是按这个公式设计的——超大参数、相对少的数据。

Chinchilla（DeepMind, 2022）：数据同样重要

Hoffmann 等人发现了 Kaplan 的实验问题：固定学习率调度导致模型没有充分收敛，低估了数据的作用。他们重新设计实验，得出结论：

> 参数和数据同等重要——50% 的额外算力给参数，50% 给数据。

公式修正为：N_opt ∝ C^0.50，D_opt ∝ C^0.50。最优 tokens/参数比约 **20**，而不是 Kaplan 说的 1–2。

直接影响：Chinchilla 70B 用 1.4 万亿 tokens 训练，以 1/4 的参数量超越了 GPT-3 175B。LLaMA 系列严格遵循 Chinchilla 公式（LLaMA 65B 用了约 22 tokens/参数），13B 版本在多数 benchmark 上超越 GPT-3 175B。

两个结论差 10 倍，谁的锅?

Kaplan 的方法问题有三个：①使用了固定学习率调度（模型没有充分训练），②主要在小尺度上做外推，③没有充分探索 tokens/参数比的极端组合。

Chinchilla 之后：Over-training 成为新常态

LLaMA 3 8B 用了约 15T tokens——约 **1,875 tokens/参数**，远超 Chinchilla 最优的 ~20。为什么？因为高质量预训练数据几近耗尽，与其找更多数据，不如用更多 epoch 重复训练——这就是"over-training"。小模型尤其受益于这种方式。

一句话总结：Scaling Laws 从"参数优先"修正到"参数数据均衡"，再到"小模型 over-training 有惊喜"。它告诉行业的不是"具体怎么配"，而是"配比有规律可循，投入有可预测的回报"——这才是 Transformer 统治力的商业根基。

十、训练与推理：Transformer 的双面人生

很多人学了整体架构，但搞不清训练和推理时发生了什么**本质不同**。这是面试最常踩的坑。

训练时（Teacher Forcing）

所有 token 同时进入模型，一次前向算出所有位置的预测。因为 Ground Truth 已知，模型可以并行计算：

```
输入:  "[BOS] the cat sat"
目标:  "the cat sat [EOS]"
一次前向，对所有 token 位置求交叉熵 loss
用 Masked Self-Attention 防止偷看未来（attention 上三角全部 −∞）
```

推理时（自回归生成）

模型只有一个初始 token "[BOS]"，逐字生成：

```
t=1: 输入 [BOS]                → 输出概率分布 → 采样 "the"
t=2: 输入 [BOS, the]           → 输出概率分布 → 采样 "cat"
t=3: 输入 [BOS, the, cat]      → 输出概率分布 → 采样 "sat"
t=4: 输入 [BOS, the, cat, sat] → 输出概率分布 → 采样 [EOS] → 停止
```

每一步都要重新跑完整前向。生成 N 个 token 要走 N 次模型——这就是为什么长生成慢。

KV Cache：推理加速的核心

每次新 token 进来，之前所有 token 的 K 和 V 要重新算——纯属浪费。

KV Cache：**把每一步的 K、V 保存在显存里**，下一步直接用缓存，只算新 token。

代价：KV cache 吃显存。LLaMA 70B 在 8K token 时，KV cache 约 20GB，加上模型本身 140GB（FP16），一张 H100 80GB 就爆了。

所以 GQA、MQA、MLA 这些注意力压缩方案的核心目标是**缩减 KV cache**，而不是缩减计算量。推理优化工程师心里的瓶颈排序：显存带宽 > 计算吞吐 > 模型大小。

十一、分词：Transformer 怎么「识字」

Transformer 处理的不是单词、不是汉字，是 **token**。Tokenizer 把原始文本切成 token 序列。

BPE（Byte Pair Encoding，字节对编码） 是主流方案：从单个字符开始，反复统计频率最高的相邻字符对，合并成新 token。高频词自成 token（"the" → 1 token），低频词拆成子词（"transformer" → "trans" + "form" + "er"）。

中文分词一般用字符级 BPE 或 SentencePiece。一个汉字通常 1–2 个 token，一个英文单词也是 1–2 个 token。

为什么这个小细节能决定模型天花板?

- **感知粒度**："pheromone"被切成 phe+rom+one，模型就无法感知"信息素"这个概念
- **模型成本**：词表 32K vs 128K，embedding 层参数差 4 倍
- **跨模型不通用**：LLaMA 和 GPT-4 的 tokenizer 切出的 token 不同，不能混用
- **语言偏见**：同样的意思，中文需要的 token 数往往比英文多，推理成本更高

十二、激活函数与归一化：被忽略的"螺丝钉"

这两样东西在架构图里不起眼，但选错了模型根本训不动。

激活函数：FFN 的开关

激活函数是 FFN 两层线性变换之间的非线性"开关"。没有它，两个矩阵乘法等价于一个，再多层也等于一层。

函数	公式	特点	现状
ReLU	max(0, x)	最简单的开关，负值一概压零	早期标准，已淘汰
GELU	x·Φ(x)	更平滑的 ReLU，负值不完全压零	BERT/GPT-2/3 时代主流
SwiGLU	x·σ(xW)·V	门控 + 线性组合，两个分支独立	**当前标准**：LLaMA/Qwen/DeepSeek 全在用

SwiGLU 虽然参数量比普通激活多 50%（多了一个门控分支的权重），但收敛更快、最终性能更好。

归一化：把数值拉回安全区

深层网络训练中，每一层的输出数值会逐渐偏移（内部协变量偏移），导致训练不稳定。

LayerNorm：把每一层的输出"标准化"——减去均值、除以标准差，再乘以可学习的缩放/平移参数。原始论文放在 Attention/FFN 之后（Post-LN）。

RMSNorm：LayerNorm 的简化版。研究发现均值偏移不重要，只做 scaling 就够了，节省约 7% 的计算。

Pre-LN vs Post-LN：Post-LN（归一化放在子层后面，原始论文用）中，回传的调整信号要绕更长的路，层数一多容易信号爆炸。Pre-LN（归一化放前面）让信号有一条"直通车道"——回传时畅通无阻。现在所有大模型统一用 Pre-RMSNorm。

十三、注意力复杂度危机与 FlashAttention

$O(n^2)$ 为什么是危机

注意力矩阵 QK^T 是 seq_len × seq_len，2048 token 序列的注意力矩阵是 2048 × 2048 = 4M 元素，1M token 就是 1 万亿元素。显存根本放不下。

为什么瓶颈在 IO 而不在计算

GPU 有两层存储：

- **HBM**（高带宽显存）：容量大（40-80GB），但慢——QKV 矩阵和模型参数都在这
- **SRAM**（片上缓存）：速度快 10 倍，但只有 20MB

原始 attention 计算中，QK^T 的中间结果反复在 HBM 和 SRAM 之间搬运——**80% 的时间花在搬运上，GPU 核心在空转**。

FlashAttention：算法不变，重排顺序

Tri Dao（斯坦福，2022）的核心创意：用 **tiling** 把 Q、K、V 切成小块，每次搬一小块到 SRAM 里一口气算完。用 **online softmax** 解决了分块 softmax 需要全局 max 的数学难题。

结果：输出和原始 attention **完全一样**（精确到 bit），但速度快 2–4 倍，内存从 $O(n^2)$ 降到 $O(n)$。这是一个"工程创新超越算法创新"的经典案例。

FlashAttention v2/v3

- **v2（2023）**：改进并行化策略，A100 FP16 达到 ~230 TFLOPS（万亿次浮点运算/秒，约 70% 理论峰值）
- **v3（2024）**：深度适配 H100 的 Hopper 架构特性（WGMMA 指令、TMA 异步拷贝），FP16 达到 740 TFLOPS、FP8 达到 ~1.2 PFLOPS（千万亿次/秒）

十四、注意力压缩三部曲：MQA → GQA → MLA

KV Cache 是推理瓶颈。缩减它的方法经历了三阶段演进：

MQA（Multi-Query Attention, Shazeer 2019）

所有 attention head **共享同一套 K、V**，只保留独立的 Q。KV Cache 降为原来的 1/h（h 是头数，比如 8 头就降为 1/8）。

代价：所有头用同样的 K、V，注意力多样性降低，有精度损失。GPT-4 和 PaLM 用了它。

GQA（Grouped-Query Attention, Ainslie 2023）

折中方案：把 h 个头分成 g 组，每组共享一套 K、V。比如 8 头分 4 组 → KV cache 降为 1/4。精度损失远小于 MQA。LLaMA 3、Qwen、Mistral 全用 GQA。

MLA（Multi-Head Latent Attention, DeepSeek 2024）

不开玩笑的创新：发现多个 head 的 K、V 之间有大量重叠信息，把 KV 投影矩阵做**低秩分解**（W_K ≈ W_K_down × W_K_up），信息空间远小于参数空间。KV Cache 降 ~10 倍，精度匹敌 GQA 甚至更好。

MLA 向行业发出的信号：**GQA 不是压缩的尽头**。低秩压缩利用了 heads 之间的结构冗余（信息论原理），而 GQA 只是把 head 粗暴分组（工程折中）。Mistral Large 3 已经跟进 MLA。

十五、MoE-Transformer 入门：把 FFN 拆成专家团队

核心动机

前面说过：FFN 是一个 key-value 记忆库，占了三分之二参数。但每个 token 经过时，FFN 的所有参数都被激活——就像每次查资料都要翻遍整个图书馆。

MoE（Mixture of Experts）的核心想法：**把一个大 FFN 拆成 N 个小 expert，每个 token 通过门控网络只挑 1-2 个最相关的 expert 计算。**

直观比喻

- **Dense Transformer**：所有员工同时干所有活儿，每个人都在用
- **MoE-Transformer**：一个前台（Router）接到任务后，判断难度和专业领域，只把任务分给最相关的 2 个专家。其他人不打扰

关键组件

- **Experts**：每个 expert 是一个独立的小 FFN
- **Router/Gate**：一个小网络，计算"这个 token 应该去哪些 expert？"，输出每个 expert 的得分
- **Top-k 选择**：只取得分最高的 k 个 expert（通常 k=1 或 2），其余忽略
- **稀疏激活**：虽然模型总参数巨大（DeepSeek V3 671B），但每个 token 只激活 5.5%（37B）

为什么是"免费午餐"

MoE 解耦了"模型知道多少"（总参数 = 硬盘）和"每次推理花多少钱"（激活参数 = RAM）。总参数可以翻几倍，推理成本几乎不变。

核心挑战：负载均衡

如果 Router 偷懒，把所有 token 都发给同一个 expert，其他 expert 闲置——MoE 就退化成了 Dense，甚至更差（因为训练不稳定）。

解决方案：

- **辅助损失**（传统）：加一个惩罚项，强迫 token 均匀分配
- **Expert Capacity**：每个 expert 有处理上限，超额的 token 直接丢弃
- **aux-loss-free**（DeepSeek V3）：用可学习的 per-expert bias 自动调节，不加辅助损失

十六、长上下文与推理优化

长上下文的三大瓶颈

1. **注意力 $O(n^2)$**：1M token 的 QK^T 矩阵有 1 万亿元素
2. **KV Cache 爆炸**：1M context 的 KV cache 几百 GB

3. **位置编码外推**：训练 4K、推理 1M，位置分布完全不同

解决路线

Ring Attention（序列并行）：1M context 一个 GPU 放不下怎么办？把序列切成多段，N 个 GPU 各算一段的注意力，通过环状通信交换 K、V 块。每个 GPU 最后获得完整注意力结果。

PagedAttention（vLLM）：KV cache 的"虚拟内存"。不再把 KV cache 作为一整块连续显存分配，而是切成固定大小的 page，按需分配。解决了显存碎片和预留过大的问题，吞吐量提升 2–4 倍。

Speculative Decoding（投机解码）：用小模型（draft model）快速生成几个候选 token，大模型（target model）一次性验证这些候选。小模型快但不准，大模型准但慢，用"先猜后验"的方式，2–3x 加速而不损失质量。

Multi-Token Prediction（MTP, DeepSeek V3）：每步不只预测下一个 token，还预测下下个 token。第二个 token 接受率 85–90%，推理时可以用来做 speculative decoding。

Continuous Batching（连续批处理）

传统批处理是"等一批请求到齐→一起推理→返回结果"。如果有一个请求来得慢，所有请求都等着——浪费时间。Continuous Batching 把推理过程拆成更细的粒度：每个请求的 attention 计算步长不一样，先算完的先返回，新请求随时插入空位。vLLM、TensorRT-LLM 等推理引擎都靠这个技术把吞吐量提了 2–5 倍。

Prefix Caching（前缀缓存）

很多请求共享相同的 prompt 前缀（如 system prompt、few-shot 示例）。Prefix Caching 把前缀部分算好的 KV cache 缓存下来，新请求来时直接复用，不用重新计算。效果取决于前缀命中率——在线客服（所有请求 system prompt 相同）命中率极高，随机用户 query 则低很多。vLLM 的 Automatic Prefix Caching 自动管理这个缓存。

vLLM 是做什么的

vLLM 是当前最主流的推理框架，核心贡献是把上面几件事整合到一起：PagedAttention 解决显存碎片、Continuous Batching 提高吞吐、Prefix Caching 减少冗余计算。加上量化支持（后面会讲），一张 A100 80GB 可以同时服务数十个 LLaMA 70B 请求。它的思路是**"不降低模型质量的前提下，把硬件用得更狠"**。

RAG（Retrieval-Augmented Generation，检索增强生成）

RAG 是"大模型查资料再回答"的经典模式：模型收到问题后，先去外部数据库（向量数据库/搜索引擎）检索相关文档，把检索结果拼进上下文，再让模型基于这些材料生成答案。

```
用户提问 → 检索（向量 DB/搜索引擎）→ 找到 top-k 相关文档
          → 文档拼进 prompt → 模型基于上下文生成答案
```

RAG 的核心价值在**解耦"记忆"和"推理"**：模型不用把全部知识塞进参数里（那需要万亿参数），而是把知识存在外部数据库，需要时实时检索。知识更新只需更新数据库，不用重新训练模型。

RAG 也有其局限性——本质上是"先检索、再生成"的两阶段流水线，检索召回的质量直接决定了生成效果。如果检索到的文档不相关，模型再聪明也会答错。这是 RAG 系统的核心瓶颈，也是为什么企业级部署往往要在检索层投入巨大精

力优化索引、重排序（reranking）和混合检索策略。

十七、训练精度：FP16、BF16、FP8 和混合精度

为什么要关心精度

模型训练的本质是大量浮点数运算。精度越高（FP32），结果越精确但越慢、越吃显存。精度越低（FP16/FP8），速度越快但可能溢出或丢失小值。

各精度对比

格式	位数	动态范围	用途	特点
FP32	32	极大	早期训练	慢、吃显存，基本被淘汰
FP16	16	较小	混合精度训练	快，但小梯度可能变零，容易溢出
BF16	16	同 FP32	**当前主流训练**	和 FP32 同样的范围、一半精度，不溢出
FP8	8	小	前沿训练（DeepSeek V3）	需块级缩放防溢出，训练成本降 40%+

混合精度训练

不是全局用 FP16，而是：大部分计算用 FP16/BF16 加速，关键精度敏感步骤（如 loss scaling、weight update）保持 FP32。这样既快又不丢精度。

DeepSeek V3 的 FP8 突破

DeepSeek V3 是业界首个大规模 FP8 训练的 SOTA 模型。核心技巧是**块级量化**——不全局用一套缩放因子，而是把 tensor 分块，每块独立缩放，避免异常值污染整块。结合 DualPipe 通信计算重叠，MFU（模型算力利用率）达到 ~70%。

推理量化：INT8 与 INT4

训练之外，**推理时也可以用更低精度**——而且做法完全不同。

训练量化是为了减少计算开销、省显存。推理量化是为了**把模型塞进更便宜的硬件、跑得更快**。一张 80GB 的 A100 只能塞一个 FP16 的 70B 模型，量化到 INT4 后变成 ~35GB，一张 48GB 的 L40S 就能跑，成本降一半。

权重量化（Weight-only Quantization）：只压缩模型参数，不压缩计算过程。计算时先把 INT4 权重解回 FP16，再算。好处是简单、显存省得多，坏处是计算速度提升有限（因为解压也要时间）。代表方案：**GGUF**（llama.cpp 用的格式，消费级显卡/CPU 都能跑）、**GPTQ**（逐层校准量化，精度保持好，但不适合 dynamic batching）、**AWQ**（按 activation 分布分配量化精度，敏感 channel 多留几位，效果跟 GPTQ 相当但硬件友好）。

KV Cache 量化：上一节讲过 KV cache 占推理显存大头。把 KV cache 从 FP16 降到 INT8，显存直接减半。精度损失极小（因为 attention 对噪声容忍度高），几乎所有生产推理引擎都默认启用。

训练后量化（PTQ） vs 量化感知训练（QAT）：PTQ 是模型训完后直接压——拿几百条校准数据算一下缩放因子就行，快但大模型精度可能掉 1-2%。QAT 在训练阶段就模拟量化误差，让模型学会适应低精度——精度几乎不掉，但需要重训或 LoRA 微调。主流做法是先用 PTQ 快速验证，精度不够再上 QAT。

量化不是"免费午餐"：INT4 比 INT8 省一半显存，但精度掉得更快，尤其是小模型（<7B）和小语种。70B+ 模型对量化的容忍度更高，因为参数冗余多。

十八、In-context Learning 与推理时计算

前面讲训练和推理时，提到过 GPT-3 的一个重要发现：模型不需要 fine-tune，给几个例子就能做任务。这背后是一整套关于"怎么用模型"的认知革命。

In-context Learning（上下文学习）

传统 NLP 流程：预训练 → 在下游任务数据上 fine-tune → 部署。每个任务要单独 fine-tune 一次。

GPT-3 改变了这个范式：**把任务说明和几个示例直接写在 prompt 里，模型照做，不需要更新任何参数。**

- **Zero-shot**：只给任务描述，不给示例。"翻译成法语：Hello →"——模型直接输出 "Bonjour"
- **Few-shot**：给 3-5 个示例。"英语：Hello → 法语：Bonjour / 英语：Goodbye → 法语：Au revoir / 英语：Thank you →"

为什么能工作？预训练过程中，模型在海量文本里已经见过了翻译、摘要、分类等各种任务的"格式"。你的 prompt 只是触发了这些隐含能力。

Chain-of-Thought（思维链）

2022 年，Google 发现：如果在 few-shot 示例里不只给"题目→答案"，而是"题目→逐步推理→答案"，模型对数学和逻辑题的准确率大幅提升。

```
❌ "15 + 27 = 42"
✅ "15 + 27 = ? 先算 15 + 20 = 35, 再加 7 = 42"
```

CoT 的本质是**把隐性推理过程外化为显性文本**。Transformer 的注意力机制天然适合这种"看着自己的思考过程继续思考"的模式——每生成一步推理，就为下一步提供了新的上下文。

推理时计算 Scaling（Inference-time Compute Scaling）

2024-2025 年最大的趋势：**让模型推理时多算一会儿，比多训几个月更划算。**

OpenAI o1/o3、Claude Extended Thinking、Gemini Deep Think 都在做同一件事——推理时不直接给答案，而是内部思考、验证、修正，最终输出。DeepSeek-R1 甚至只靠强化学习（没加任何 supervised 数据）就让模型自己学会了

CoT。

这意味着 Transformer 的能力增长进入了双轨时代——预训练（学知识）+ 推理时计算（用知识）。两个维度可以同时 scale。

十九、后训练：RLHF、DPO 与对齐

预训练完的模型能续写文本，但它不知道"什么该说、什么不该说"。它可能生成有害内容、编造事实、或拒绝回答正常问题。

后训练（Post-training） 解决的就是这个问题：让模型的输出符合人类的期望和价值观。

RLHF（Reinforcement Learning from Human Feedback）

三步走：

1. **SFT（Supervised Fine-Tuning）**：找人来写一批"理想回答"，用这些数据微调模型。让模型先学会"好回答长什么样"
2. **训练奖励模型（Reward Model）**：让模型对同一个问题生成多个回答，人工标注哪个更好。用这些偏好数据训练一个"打分器"
3. **PPO 强化学习**：用奖励模型给模型生成的回答打分，通过强化学习优化模型让它"写高分答案"

RLHF 是 ChatGPT/GPT-4/Claude 的训练核心。但工程非常复杂——奖励模型会作弊、PPO 训练不稳定、需要大量人工标注。

DPO（Direct Preference Optimization）

2023 年斯坦福提出的更简单方案：**不需要单独训练奖励模型，直接用好/坏回答的对比来优化模型。**

DPO 把 RLHF 的三个步骤压缩成了一个——直接从偏好数据中调整模型，让好回答的概率更高、坏回答的概率更低。工程开销大幅降低，效果相当。

GRPO（Group Relative Policy Optimization）

DeepSeek-R1 用的方案，更进一步简化：**一组回答互相对比，不需要外部奖励模型，也不需要人类标注。**

同一个问题生成多个回答，把得分最高的那个当"好样本"、最低的当"坏样本"，用差距来训练模型。DeepSeek-R1 完全用 GRPO（没加任何 SFT 数据）就让模型自发学会了 CoT 推理。

后训练的趋势

从"需要大量人工标注"（RLHF）→ "只需要偏好数据"（DPO）→ "AI 自己对比自己的回答"（GRPO），后训练正在越来越自动化。这跟预训练数据的趋势一致——合成数据替代真实数据，AI 自己教自己。

二十、替代架构探秘：SSM、Mamba 与混合路线

Transformer 统治了九年，但 $O(n^2)$ 的注意力复杂度一直是达摩克利斯之剑。2023 年起，一系列 O(n) 或 O(n log n) 的替代架构开始认真挑战 Transformer。

SSM（State Space Model，状态空间模型）

SSM 源自信号处理和控制论——用一组微分方程描述系统状态如何随时间演化。在 NLP 的语境里：一个 SSM 层维护一个"记忆状态"，每读一个 token 就更新这个状态。因为是逐 token 更新的，所以是 O(n)。

传统 SSM 的问题是状态更新跟输入无关（线性时不变），表达能力不足。

Mamba：选择性 SSM

Gu & Dao（2023）的关键贡献：**让 SSM 的参数取决于输入**——不同 token 看到的信息不同，"什么该记住、什么该忘"是基于内容的选择。这就是"选择性"的含义。

Mamba 在 1–3B 参数级别证明了自己——跟同规模的 Transformer 性能相当，但推理吞吐高 5–6 倍，因为不需要存 KV cache。

Mamba–2（2024）进一步把 SSM 和线性注意力在数学上统一了——证明了"好的 SSM 本质上也是一种注意力"。

混合架构：不是替代，是拼接

2025 年后，纯替代架构路线慢慢让位给了混合路线。因为大家发现：

- **Attention 长于精密推理**（需要精确的 token–to–token 交互）
- **SSM/线性模型长于长序列扫描**（O(n) 处理海量上下文）

Qwen3.5 把 75% 的 attention 层换成了 Gated DeltaNet（一种线性注意力），剩下的 25% 保留标准 attention 用于关键推理步骤。Jamba、Zamba 也在走类似路线。

趋势判断：未来 2–3 年，最可能出现的不是"Transformer 被某一种架构取代"，而是 **Transformer + X 的混合架构找到最优配比**。

补充视角：Hyperscale Lottery（超大规模彩票）

Transformer 的统治地位还有一个不易察觉的原因：**它跟硬件深度绑定了**。

Sara Hooker 提出的 "Hyperscale Lottery" 理论指出：Transformer 至今仍是绝对统治者，不完全是因为它数学上最优，而是因为 NVIDIA 的 GPU 经过十年演化，矩阵乘法（GEMM）被极致优化、CUDA 生态围绕它构建、FlashAttention 等全部为它量身定制。整个研究–工程–硬件生态被锁死在 Transformer 上。

这意味着两件事：第一，替代架构不仅要算法好，还要等硬件支持成熟（Blackwell B200 已开始原生支持 SSM/卷积操作，这是一个转折信号）；第二，我们看到的"Transformer 的胜利"，有一部分是硬件红利和生态锁定的产物，而非架构本身不可替代。

这给"替代架构"的讨论提供了一个现实注脚：**架构之争不仅是数学问题，也是生态问题。**

二十一、多模态 Transformer：ViT 与图文模型

Transformer 不只是文本模型。2020 年以来，它被证明在图像、视频、音频上同样有效。

ViT（Vision Transformer）

2020 年 Google 提出：**把图片切成 16×16 的 patch（小方块），每个 patch 当成一个 token，直接扔进 Transformer。**

不需要 CNN 的卷积核、池化层——纯 Transformer 在足够大的数据量下（ImageNet-21K+）超越了当时的 CNN SOTA。这件事传递的信息和 NLP 领域一样："Transformer 的通用性远超我们想象"。

多模态模型：图文融合

既然文本和图像都能用 Transformer 编码成向量，自然就可以在向量空间里做"对齐"：

- **CLIP**（OpenAI, 2021）：双塔架构——一个 Transformer 编码图片（ViT），一个 Transformer 编码文本。用对比学习让匹配的图文向量接近、不匹配的远离。训练完后可以直接做零样本分类（"这张图里有猫吗？"）
- **Flamingo / GPT-4V / Gemini**：更深的融合——图片的 patch token 和文本 token 交错排列，交给一个统一的 Transformer 处理，输出可以是文本、代码或其他模态

多模态 Transformer 证明了同样的"按需分配 + 并行处理"配方在多个感知模态上都有效——底层架构通用性越强，不同模态之间越容易对齐。

二十二、一张图谱串起来

```
文本输入
  ↓ Tokenizer → 分词成 token 序列
  ↓ Embedding + Positional Encoding → 向量表示
  ↓
  ↓ [开始 1 层 Transformer]
  ↓   ├─ Pre-RMSNorm（归一化稳定训练）
  ↓   ├─ Multi-Head Self-Attention（QKV → 加权信息聚合）
  ↓   │    ├─ 多头检测多种关系模式
  ↓   │    ├─ FlashAttention 加速（tiling + online softmax）
  ↓   │    └─ GQA/MQA/MLA 压缩 KV cache
  ↓   ├─ Add（残差连接）
  ↓   ├─ Pre-RMSNorm
  ↓   ├─ FFN（Key-Value 记忆库 → 提取知识）
  ↓   │    ├─ SwiGLU 激活
  ↓   │    └─ MoE：拆成 N 个 expert，Router 只激活 top-k
  ↓   └─ Add
  ↓ [重复 N 层，N 通常 12-120]
  ↓
  ↓ 输出层 → 映射到词表 → softmax → 采样下一个 token
```

二十三、与 MoE 前置知识的对照

这份文档和 MoE 前置知识文档是**镜像设计**。对照阅读可以加深两条线的理解：

概念	Transformer 视角	MoE 视角
软路由	Attention 用 QK 相似度路由信息	Gate/Router 用门控网络路由计算
并行处理	多头注意力并行检测多种关系	多个 Expert 并行处理不同知识域
专业化	不同 head 学习不同依赖模式	不同 expert 专注不同领域知识
压缩冗余	MLA 低秩压缩 KV cache	细粒度专家减少参数冗余
条件计算	Cross-Attention 按需检索源句	Top-k 路由只激活最相关 expert
记忆架构	FFN = Key-Value 记忆库	每个 Expert = 独立知识分区
负载均衡	GQA 分组共享 K/V	Aux Loss / Expert Capacity / aux-loss-free
精度演进	FP16 → BF16 → FP8 混合精度	→ 同样受益于精度优化

两者共同的核心命题：**让每个 token 更精准地获得它需要的信息（Transformer）和需要的计算（MoE）**。

二十四、延伸阅读

- Transformer 横纵分析报告 — 纵向历史 + 横向竞争 + 10道面试题 + 5道场景题 + 3道应用题
- MoE 前置知识 — MoE 混合专家模型的入门指南（如有）

核心论文

- Vaswani et al., "Attention Is All You Need" (2017) — https://arxiv.org/abs/1706.03762
- Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate" (2014) — https://arxiv.org/abs/1409.0473
- Brown et al., "Language Models are Few-Shot Learners" (GPT-3, 2020) — https://arxiv.org/abs/2005.14165
- Kaplan et al., "Scaling Laws for Neural Language Models" (2020) — https://arxiv.org/abs/2001.08361
- Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla, 2022) — https://arxiv.org/abs/2203.15556
- Dao et al., "FlashAttention" (2022) — https://arxiv.org/abs/2205.14135 ; v2 (2023) — https://arxiv.org/abs/2307.08691 ; v3 (2024) — https://arxiv.org/abs/2407.08614

- Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need" (MQA, 2019) — https://arxiv.org/abs/1911.02150
- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models" (2023) — https://arxiv.org/abs/2305.13245
- DeepSeek, "DeepSeek-V2" (MLA, 2024) — https://arxiv.org/abs/2405.04434 ; "DeepSeek-V3" (2024) — https://arxiv.org/abs/2412.19437
- Shazeer et al., "Outrageously Large Neural Networks" (MoE, 2017) — https://arxiv.org/abs/1701.06538
- Fedus et al., "Switch Transformers" (2021) — https://arxiv.org/abs/2101.03961
- Su et al., "RoFormer" (RoPE, 2021) — https://arxiv.org/abs/2104.09864
- Geva et al., "Transformer Feed-Forward Layers Are Key-Value Memories" (2021) — https://arxiv.org/abs/2012.14913

> **下一篇**：Transformer 横纵分析报告 — 从这里直接进入完整的纵向历史分析和横向竞争对比。